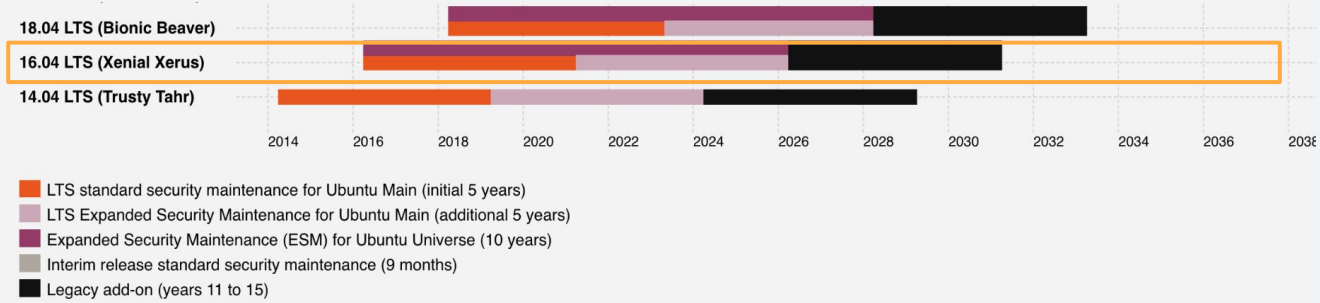
Counting Chaos

Breaking a Legacy Driver in EOL'd Ubuntu for Root

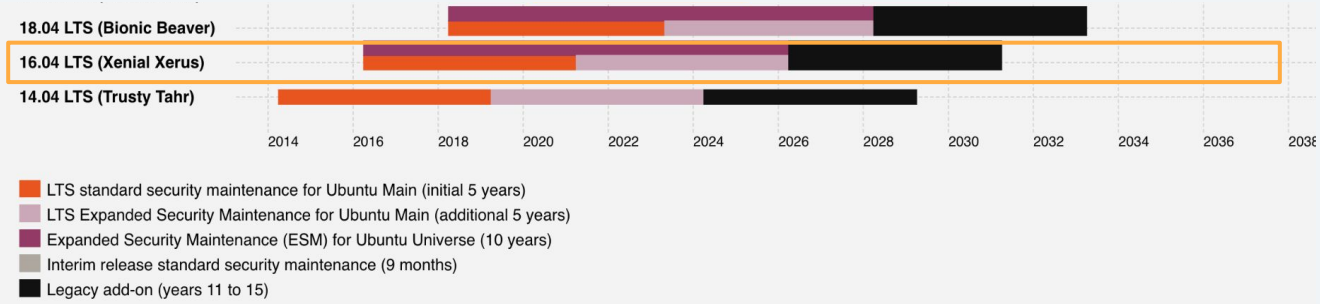
Goal: Local privesc on **Ubuntu Xenial (16.04.7 LTS)**

- **End of *standard support* in April 2021**
  - **Expanded Security Maintenance** (ESM), part of Ubuntu Pro, ending in ~~2026~~ 2035 (announced Nov 2025)



Goal: Local privesc on **Ubuntu Xenial (16.04.7 LTS)**

- **End of *standard support* in April 2021**
  - **Expanded Security Maintenance** (ESM), part of Ubuntu Pro, ending in ~~2026~~ 2035 (announced Nov 2025)
- **Demo targets kernel version 4.4.188 (also EOL)**



Casing the Joint

- **Focus only on legacy drivers that don't exist anymore, eg:**
  - aufs - legacy overlay filesystem
  - Socket families - IPX, IrDA, etc.
  - **Traffic control / scheduling** (`net/sched`)

Quick Background: Traffic control and qdiscs

- **Traffic control (tc)** manages packet shaping and scheduling, and **queuing disciplines (qdiscs)** are heuristics for *when* transmission happens, attached to an iface.
- To interact with tc, we use **netlink** (NETLINK_ROUTE family) for kernel IPC.
- **Bug found in now-deleted `sch_atm` qdisc (`net/sched/sch_atm.c`) for Asynchronous Transfer Mode (ATM) virtual circuits.**

https://starlabs.sg/blog/2023/06-breaking-the-code-exploiting-and-examining-cve-2023-1829-in-cls_tcindex-classifier-vulnerability/
https://syst3mfailure.io/rbtree-family-drama/

Vulnerability: Refcount mismanagement in `atm_tc_change` (UAF)

Method invoked through `RTM_NEWTCLASS` netlink message with these nl attributes:

```
static const struct nla_policy atm_policy[TCA_ATM_MAX + 1] =
{   [TCA_ATM_FD]        = { .type = NLA_U32 },
    [TCA_ATM_EXCESS]    = { .type = NLA_U32 },
};
```

```
static int atm_tc_change(struct Qdisc *sch, u32 classid, u32 parent,
            struct nlattr **tca, unsigned long *arg)
{
    //...
    error = nla_parse_nested(tb, TCA_ATM_MAX, opt, atm_policy);
    if (error < 0)
        return error;

    if (!tb[TCA_ATM_FD])
        return -EINVAL;
    fd = nla_get_u32(tb[TCA_ATM_FD]);

    // ...
    if (!tb[TCA_ATM_EXCESS])
        excess = NULL;
    else {
        excess = (struct atm_flow_data *)
            atm_tc_get(sch, nla_get_u32(tb[TCA_ATM_EXCESS]));
        if (!excess)
            return -ENOENT;
    }

    sock = sockfd_lookup(fd, &error);
    if (!sock)
        return error;   /* f_count++ */

    if (sock->ops->family != PF_ATMSVC && sock->ops->family != PF_ATMPVC)
{       error = -EPROTOTYPE;
        goto err_out;
    }

    //...
    flow = kzalloc(sizeof(struct atm_flow_data) + hdr_len, GFP_KERNEL);
    // ...
    flow->ref = 1;
    flow->excess = excess;
    list_add(&flow->list, &p->link.list);
    // ...
    return 0;
err_out:
    if (excess)
        atm_tc_put(sch, (unsigned long)excess);
    sockfd_put(sock);
    return error;
}
```

Normal error handling:

Lookup excess from `TCA_ATM_EXCESS`, increment refcount when found

`excess->ref++`

If wrong `TCA_ATM_FD` family, cleanup through `err_out` branch

`err_out`: decrement excess refcount, free excess if 0

`excess->ref--`

```
static int atm_tc_change(struct Qdisc *sch, u32 classid, u32 parent,
            struct nlattr **tca, unsigned long *arg)
{
    //...
    error = nla_parse_nested(tb, TCA_ATM_MAX, opt, atm_policy);
    if (error < 0)
        return error;

    if (!tb[TCA_ATM_FD])
        return -EINVAL;
    fd = nla_get_u32(tb[TCA_ATM_FD]);

    // ...
    if (!tb[TCA_ATM_EXCESS])
        excess = NULL;
    else {
        excess = (struct atm_flow_data *)
            atm_tc_get(sch, nla_get_u32(tb[TCA_ATM_EXCESS]));
        if (!excess)
            return -ENOENT;
    }

    sock = sockfd_lookup(fd, &error);
    if (!sock)
        return error;   /* f_count++ */

    if (sock->ops->family != PF_ATMSVC && sock->ops->family != PF_ATMPVC)
    {
        error = -EPROTOTYPE;
        goto err_out;
    }

    //...
    flow = kzalloc(sizeof(struct atm_flow_data) + hdr_len, GFP_KERNEL);
    // ...
    flow->ref = 1;
    flow->excess = excess;
    list_add(&flow->list, &p->link.list);
    // ...
    return 0;
err_out:
    if (excess)
        atm_tc_put(sch, (unsigned long)excess);
    sockfd_put(sock);
    return error;
}
```

Vulnerable error handling:

Lookup excess from `TCA_ATM_EXCESS`, increment refcount when found

`excess->ref++`

No `err_out` taken if TCA_ATM_FD is a bad fd

No refcount decrement!

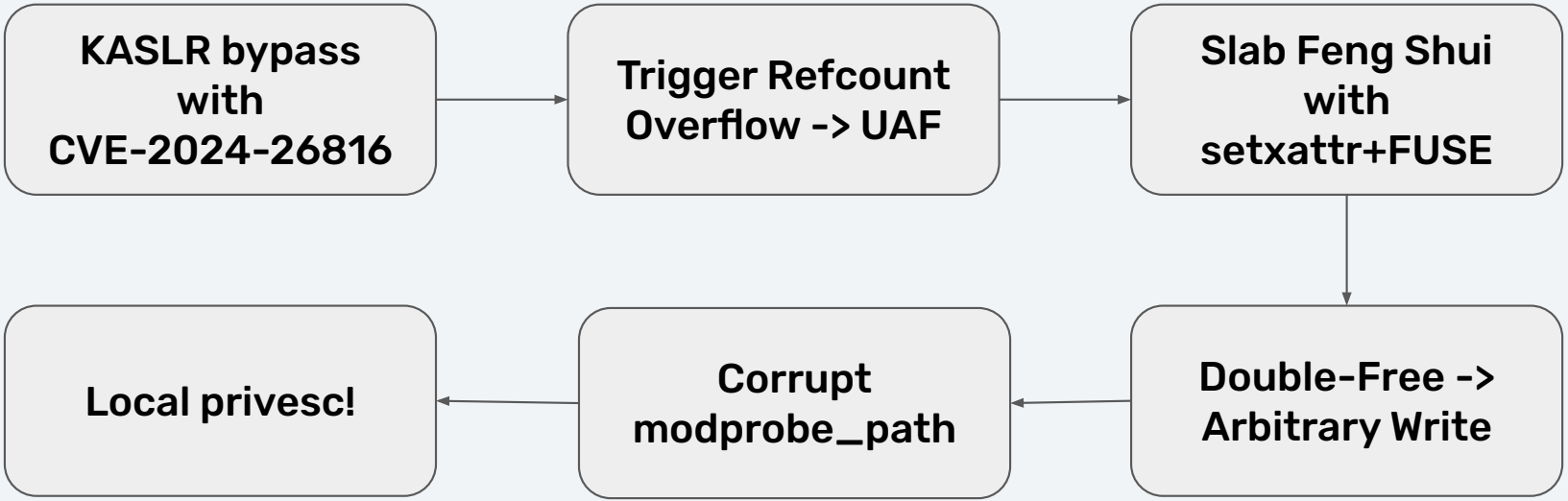
```
static int atm_tc_change(struct Qdisc *sch, u32 classid, u32 parent,
            struct nlattr **tca, unsigned long *arg)
{
    //...
    error = nla_parse_nested(tb, TCA_ATM_MAX, opt, atm_policy);
    if (error < 0)
        return error;

    if (!tb[TCA_ATM_FD])
        return -EINVAL;
    fd = nla_get_u32(tb[TCA_ATM_FD]);

    // ...
    if (!tb[TCA_ATM_EXCESS])
        excess = NULL;
    else {
        excess = (struct atm_flow_data *)
            atm_tc_get(sch, nla_get_u32(tb[TCA_ATM_EXCESS]));
        if (!excess)
            return -ENOENT;
    }

    sock = sockfd_lookup(fd, &error);
    if (!sock)
        return error;   /* f_count++ */

    if (sock->ops->family != PF_ATMSVC && sock->ops->family != PF_ATMPVC)
{       error = -EPROTOTYPE;
        goto err_out;
    }

    //...
    flow = kzalloc(sizeof(struct atm_flow_data) + hdr_len, GFP_KERNEL);
    // ...
    flow->ref = 1;
    flow->excess = excess;
    list_add(&flow->list, &p->link.list);
    // ...
    return 0;
err_out:
    if (excess)
        atm_tc_put(sch, (unsigned long)excess);
    sockfd_put(sock);
    return error;
}
```

Vulnerable error handling:

Triggering this path `UINT_MAX - 1` times overflows `excess->ref`

```
static int atm_tc_change(struct Qdisc *sch, u32 classid, u32 parent,
            struct nlattr **tca, unsigned long *arg)
{
    //...
    error = nla_parse_nested(tb, TCA_ATM_MAX, opt, atm_policy);
    if (error < 0)
        return error;

    if (!tb[TCA_ATM_FD])
        return -EINVAL;
    fd = nla_get_u32(tb[TCA_ATM_FD]);

    // ...
    if (!tb[TCA_ATM_EXCESS])
        excess = NULL;
    else {
        excess = (struct atm_flow_data *)
            atm_tc_get(sch, nla_get_u32(tb[TCA_ATM_EXCESS]));
        if (!excess)
            return -ENOENT;
    }

    sock = sockfd_lookup(fd, &error);
    if (!sock)
        return error;   /* f_count++ */

    if (sock->ops->family != PF_ATMSVC && sock->ops->family != PF_ATMPVC)
{       error = -EPROTOTYPE;
        goto err_out;
    }

    //...
    flow = kzalloc(sizeof(struct atm_flow_data) + hdr_len, GFP_KERNEL);
    // ...
    flow->ref = 1;
    flow->excess = excess;
    list_add(&flow->list, &p->link.list);
    // ...
    return 0;
err_out:
    if (excess)
        atm_tc_put(sch, (unsigned long)excess);
    sockfd_put(sock);
    return error;
}
```

Exploit Strategy



Slab Feng Shui

- **Target chunk is an `atm_flow_data`, which is allocated in `kmalloc-128+` (variable-sized)**
- **To refill, we'll use the popular `setxattr` + FUSE technique:**
  - Nice properties: control size of allocation, and all of its contents
  - Custom FUSE VFS helps prevent allocation from being freed in same syscall path

https://www.willsroot.io/2022/01/cve-2022-0185.html
https://chomp.ie/Blog+Posts/Put+an+io_uring+on+it+-+Exploiting+the+Linux+Kernel

What to target with our UAF?

Corrupting `list_head` field gives us *arbitrary write* during unlinking!

```
struct atm_flow_data {
    struct Qdisc_class_common common;
    struct Qdisc        *q; /* FIFO, TBF, etc. */
    struct tcf_proto __rcu  *filter_list;
    struct tcf_block    *block;
    struct atm_vcc      *vcc;   /* VCC; NULL if VCC is closed */
    void            (*old_pop)(struct atm_vcc *vcc,
                               struct sk_buff *skb); /* chaining */
    struct atm_qdisc_data   *parent;    /* parent qdisc */
    struct socket       *sock;      /* for closing */
    int         ref;        /* reference count */
    struct gnet_stats_basic_sync    bstats;
    struct gnet_stats_queue qstats;
    struct list_head    list;
    struct atm_flow_data    *excess;    /* flow for excess traffic;
                                NULL to set CLP instead */
    int         hdr_len;
    unsigned char       hdr[];      /* header data; MUST BE LAST */
};
```

List unlinking to arbitrary write

```
static void atm_tc_put(struct Qdisc *sch, unsigned long cl)
{
    struct atm_qdisc_data *p = qdisc_priv(sch);
    struct atm_flow_data *flow = (struct atm_flow_data *)cl;
    pr_debug("atm_tc_put(sch %p,[qdisc %p],flow %p)\n", sch, p, flow);
    if (--flow->ref)
        return;
    pr_debug("atm_tc_put: destroying\n");
    list_del_init(&flow->list);
    pr_debug("atm_tc_put: qdisc %p\n", flow->q);
    qdisc_put(flow->q);
    tcf_block_put(flow->block);
    if (flow->sock) {
        pr_debug("atm_tc_put: f_count %ld\n",
            file_count(flow->sock->file));
        flow->vcc->pop = flow->old_pop;
        sockfd_put(flow->sock);
    }
    if (flow->excess)
        atm_tc_put(sch, (unsigned long)flow->excess);
    if (flow != &p->link)
        kfree(flow);
    /*
     * If flow == &p->link, the qdisc no longer works at this point and
     * needs to be removed. (By the caller of atm_tc_put.)
     */
}
```

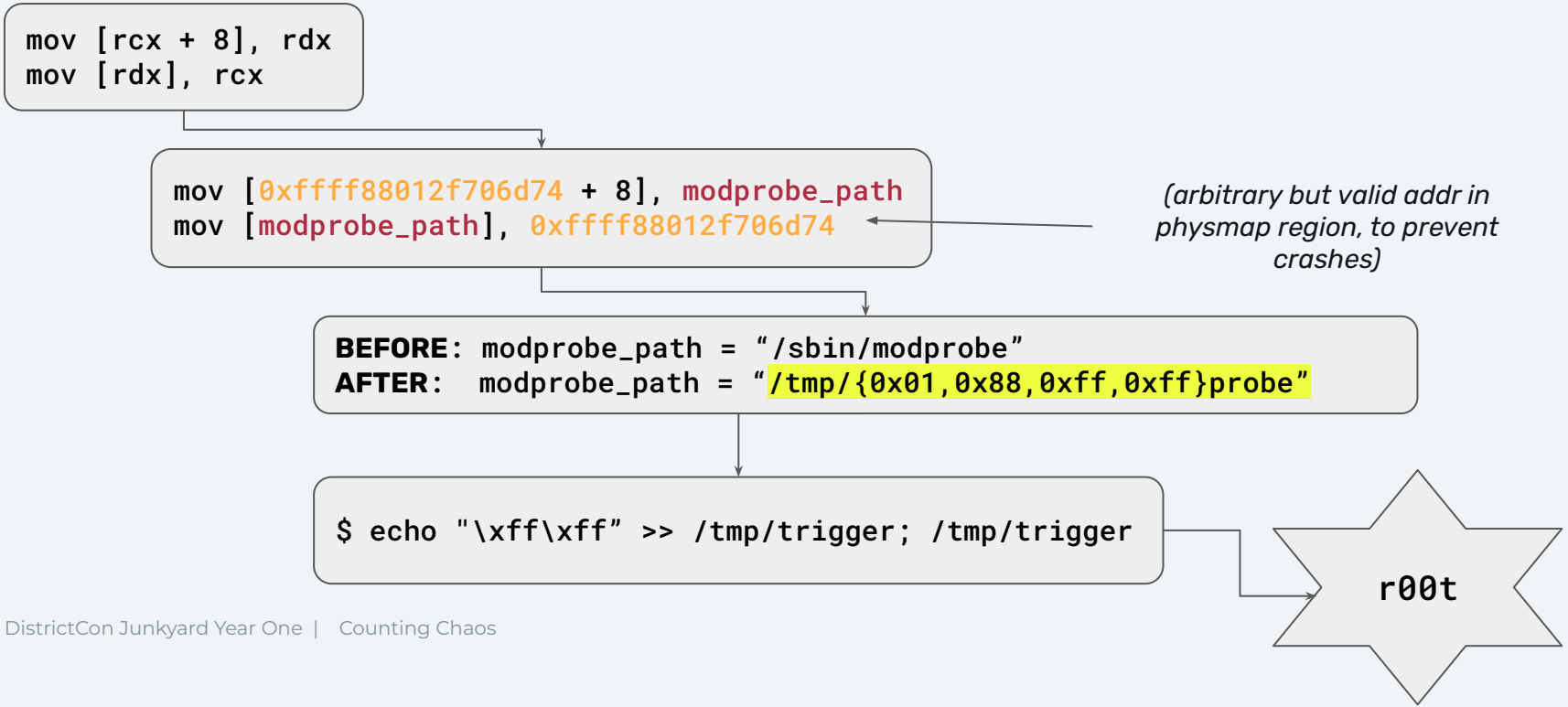
Qdisc deletion triggers double-free of chunk, triggering list unlinking

```
static void __list_del(struct list_head * prev, struct list_head * next) {
    next->prev = prev;
    WRITE_ONCE(prev->next, next);
}
```

```
mov [rcx + 8], rdx
mov [rdx], rcx
```

Exploit Strategy

modprobe_path corruption



Don’t forget the infoleak!

- **We need to know where `modprobe_path` is!**
- **Ubuntu GA kernel has no KASLR enabled by default 🤯**
- **For our exploit targeting KASLR-enabled distros:**
  - Can’t really use our bug to leak, only viable access is free path :/
  - Cheat with an “universal” n-day: **CVE-2024-26816**
    - **Leak startup_xen from world-readable /sys/kernel/notes**
    - **11+ years old bug!**

BLOCKER: It can take a long time

- **Overflowing a 32-bit refcount requires 4,294,967,296 (UINT_MAX) increments**
- **Exploit incorporates several performance optimizations:**
  - Multiple threads pinned to each CPU core, messages are batched, and aggressive code removal (shoutout to Claude)
- **Best runs are plateauing at 20 mins**
  - Backup demo will run against a sch_atm.ko with a minimal patch (`int ref -> short ref`)

Demo time 😮